OOP Solved Question Papers

2023, 2022 and 2021 Final Examinations — Exam-Oriented Solutions

Prepared for exam revision

How to use this guide
2023 — B.Sc (Engg.) 2nd Year 2nd Semester Final Examination
- Q1
- Q2
- Q3
- Q4
- Q5
- Q6
- Q7
- Q8

2022 — B.Sc (Engg.) 2nd Year 1st Semester Final Examination
- Q1
- Q2
- Q3
- Q4
- Q5
- Q6
- Q7
- Q8

2021 — B.Sc (Engg.) 2nd Year 1st Semester Final Examination
- Q1
- Q2
- Q3
- Q4
- Q5
- Q6
- Q7
- Q8

Quick Revision Checklist

How to use this guide

The three papers below (2023, 2022, 2021) share most of their topics — OOP fundamentals, classes & objects, constructors/destructors, friend functions, operator overloading, inheritance, Java basics, and file handling. Each paper is solved **in the original order of its questions**, with definitions kept exam-crisp and every programming question given as a complete, compilable C++ (or Java) program.

2023 — B.Sc (Engg.) 2nd Year 2nd Semester Final Examination

Q1

(a) What is object oriented programming? How is it different from procedure-oriented programming? (03)

Object Oriented Programming (OOP) is a programming paradigm that organizes software design around **objects** — self-contained units that bundle together data (attributes) and the functions (methods) that operate on that data — rather than around a sequence of instructions. OOP is built on four pillars: encapsulation, abstraction, inheritance, and polymorphism.

Procedure-Oriented Programming	Object Oriented Programming
Program is divided into functions; data and functions are separate	Program is divided into objects; data and functions are bound together
Data moves freely between functions	Data is hidden and accessed only through member functions
No/very limited data hiding	Data hiding via access specifiers (private/protected)
Overloading not supported	Function and operator overloading supported
Top-down approach	Bottom-up approach
Example: C	Example: C++, Java

(b) Distinguish between data abstraction and data encapsulation. (03)

- **Data abstraction** is the process of exposing only the essential features of an object to the outside world while hiding the implementation details — it deals with *what* an object does (interface).
- **Data encapsulation** is the mechanism of binding data and the functions that manipulate it together into a single unit (a class), and restricting direct access to the data — it deals with *how* the data is protected/bundled (implementation).

In short, abstraction hides complexity; encapsulation hides data. Abstraction is achieved through classes/interfaces, encapsulation through access specifiers.

(c) How are data and functions organized in an object-oriented program? (03)

In OOP, data and the functions that operate on that data are enclosed together inside a **class**. An object is an instance of the class. Data members are usually declared `private`, and are accessed/modified only through `public` member functions, which act as a controlled interface. This binding of data and functions is called encapsulation, and it prevents data from being accessed accidentally by code outside the class.

(d) Define dynamic binding. How is it useful in OOP? (03)

Dynamic binding (also called late binding or run-time polymorphism) is the linking of a function call to its function body at **run time**, rather than at compile time. In C++ it is implemented using **virtual functions** and pointers/references to a base class.

Usefulness: it allows a base-class pointer/reference to call the correct derived-class version of a function depending on the actual object it points to, enabling polymorphism, extensibility (new derived classes can be added without changing existing code), and cleaner, more maintainable designs.

(e) Write a C++ program that asks for a temperature in Fahrenheit and displays it in Celsius. (02)

```cpp
#include <iostream>
using namespace std;

int main() {
    float fahrenheit, celsius;
    cout << "Enter temperature in Fahrenheit: ";
    cin >> fahrenheit;
    celsius = (fahrenheit - 32) * 5.0 / 9.0;
    cout << "Temperature in Celsius: " << celsius << endl;
    return 0;
}
```

Q2

(a) Describe the major parts of a C++ program with example. (03)

A typical C++ program has these parts:

1. **Preprocessor directives** — e.g. `#include <iostream>`, processed before compilation.
2. **Namespace declaration** — e.g. `using namespace std;`.
3. **Class declarations** (if any) — defines user data types.
4. **Function declarations/prototypes**.
5. **The `main()` function** — the entry point; execution always starts here.

6. **Function definitions** — bodies of functions used in the program.
7. **Comments** — `//` or `/* */`, ignored by the compiler.

```cpp
#include <iostream>        // preprocessor directive
using namespace std;       // namespace

int add(int a, int b);     // function prototype

int main() {                // main function - entry point
    int x = 5, y = 10;
    cout << "Sum = " << add(x, y) << endl;
    return 0;
}

int add(int a, int b) {     // function definition
    return a + b;
}
```

(b) Develop a C++ program that employs the concept of call by address. (02)

```cpp
#include <iostream>
using namespace std;

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int x = 5, y = 10;
    cout << "Before swap: x=" << x << " y=" << y << endl;
    swap(&x, &y);
    cout << "After swap: x=" << x << " y=" << y << endl;
    return 0;
}
```

(c) Define an inline function. Develop a C++ program that employs the concept of inline function. (03)

An **inline function** is a function for which the compiler is requested (using the `inline` keyword) to insert the complete body of the function at every point the function is called, instead of performing a normal function call. This avoids the overhead of function-call mechanisms (stack push/pop, jump) and is typically used for small, frequently-called functions.